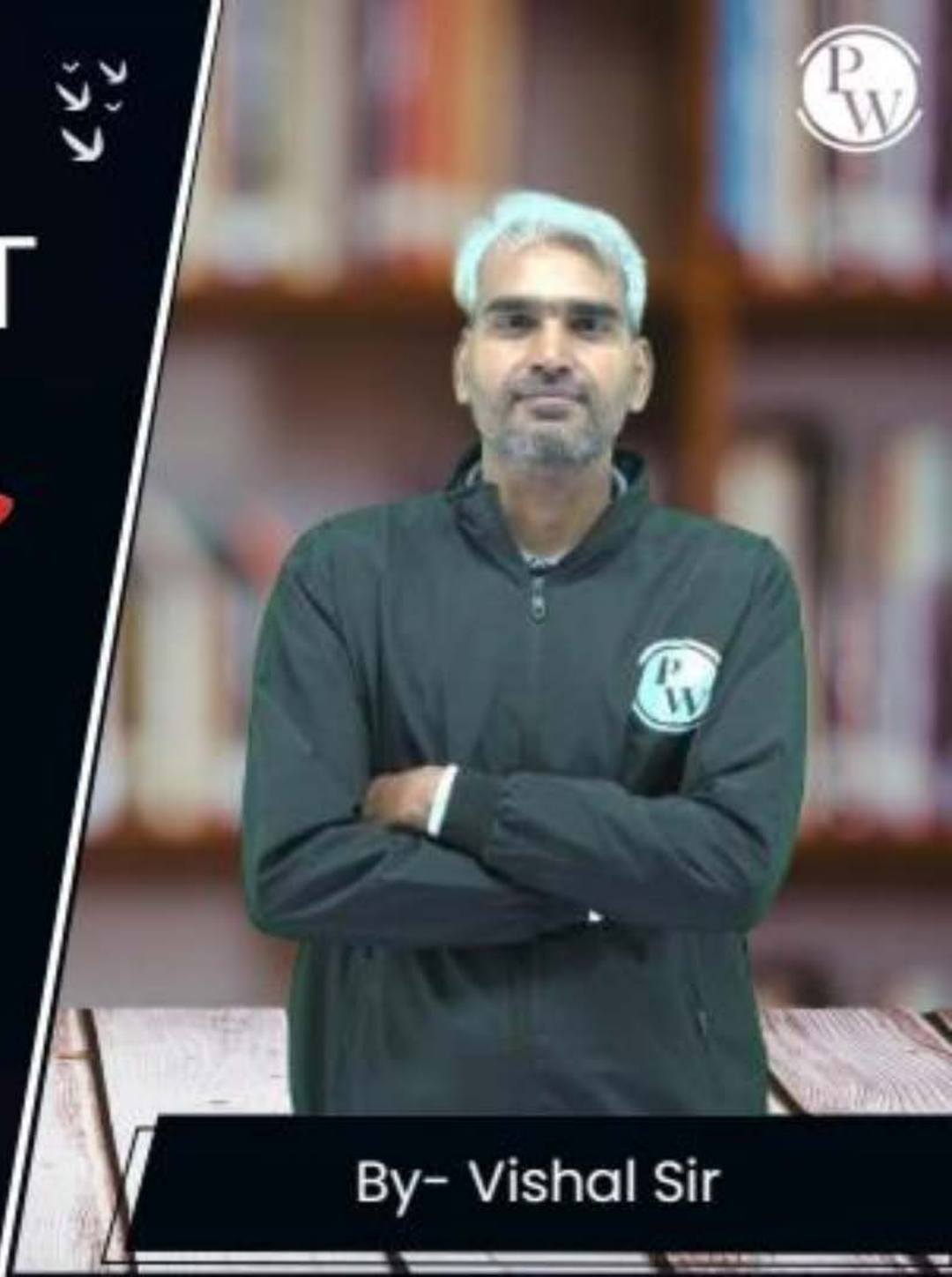
Computer Science & IT



Database Management System

Transaction & concurrency control

Lecture No. 01



By- Vishal Sir

Recap of Previous Lecture



- Topic — Practice questions
- Topic — Domain relational calculus (DRC)

Slide

Topics to be Covered



- Topic: Transaction
- Topic: ACID properties
- Topic: Atomicity
- Topic: Durability

Slide



Topic : Transaction

A sequence of logically related operations that are used to perform a particular task is called a transaction

Read (A) :- This operation is used to read the value of dataitem 'A' from database (i.e. disk) to main memory

Write (A) :- The current value of dataitem A that is present in main memory will be updated into the database

* Let 'A' and 'B' represent the amount in two different accounts. A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction

Read (A): Because of this oph a copy of dataitem A will be created in Main memory

A = A - 500; This oph will be performed only in main memory

Write (A) Current value of dataitem A from the M.M. will be updated into database

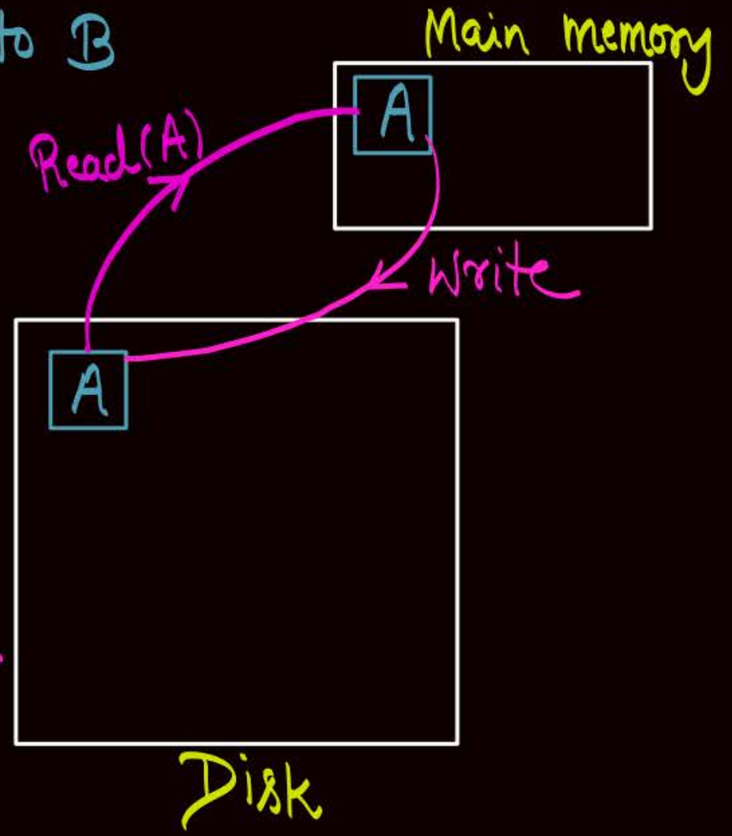
Read (B)

B = B + 500

Write (B)

Commit { Transaction Committed means transaction executed successfully, from user point of view }

These two oph are not important from database Point of view





Topic : Blind write

{ When a transaction T updates a dataitem without reading that dataitem from database }

R(A) = Read (A)
W(A) = Write (A)

Transaction (T)

R(A)
R(C)
W(A) ← It is not a blind write oph because 'T' has read the dataitem 'A' from the database before this W(A) oph

R(D)
W(B) ← It is blind write oph because transaction 'T' did not read the value of dataitem 'B' before this W(B) oph

Not a blind write → W(D)

Consider two transactions T_1 & T_2

T_1
R(A)
R(B)

T_2
R(B)
R(A)

T_1 & T_2 are two different transactions

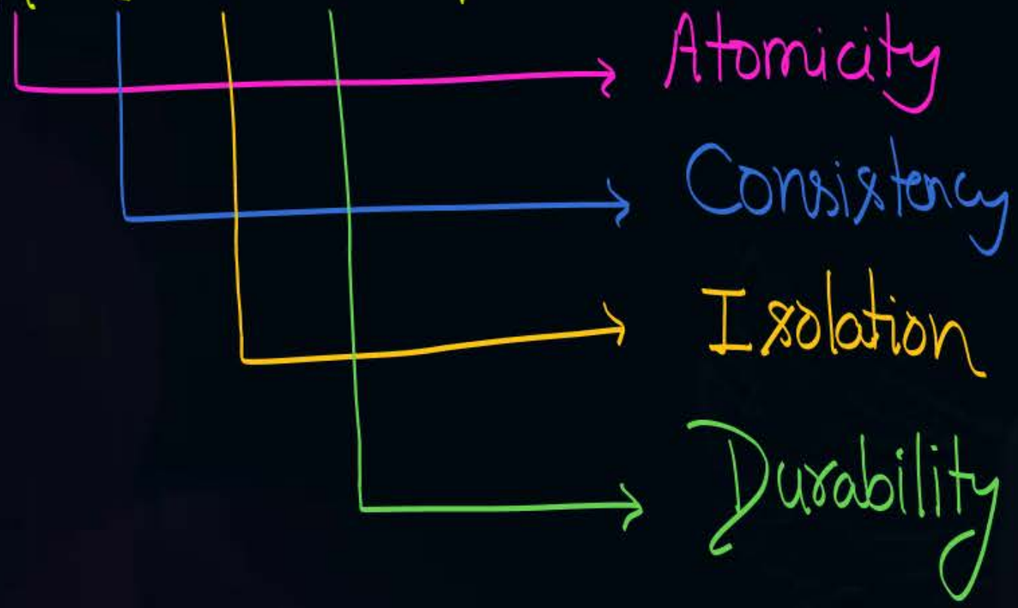
⇓

We can not change the position of the operations in the same transaction.



Topic : ACID properties

* For the integrity of the database A C I D properties must be satisfied





Topic : Atomicity

* Atomicity states that, either execute all operations of a transaction or none of them.

There are two problems

1. We can not execute all the operations of a transaction at once, they will be executed in a sequence one after another
2. If we start executing the operations of a transaction, then we can not guarantee that all operations of that transaction will get executed without any failure

* Let 'A' and 'B' represent the amount in two different accounts. A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction

Read (A)

A = A - 500;

Write (A)

Read (B)

B = B + 500

Write (B)

Commit

These two opn are not important from database Point of view

* failure occur in between

* If we start executing the operations of this transaction, and if failure occur before executing the Commit operation then atomicity is violated

{ Transaction Committed means transaction executed successfully, from user point of view }

Note :-

To ensure atomicity, if transaction failed before executing its commit operation successfully, then we must undo all the changes performed by the transaction before its failure

{ If we are able to do so, then it will be similar to the situation when none of the operation of the transaction is executed }

* The process of undoing the changes performed by a transaction is called rollbacking of that transaction

To ensure atomicity, "Recovery management Component" is used



Topic : Recovery management component

* Recovery management Component is responsible for "undo" and "redo" operation

"undo" oph will be performed w.r.t. failed transaction

"redo" oph will be performed w.r.t. Committed transaction

↳ The process of undoing the changes performed by a failed transaction is rollback

Note:- For the purpose of "undo" and "redo" operation transaction log is maintained by recovery management Component



Topic : Transaction log

- Transaction log is used to record the activities performed by the transactions

* Let 'A' and 'B' represent the amount in two different accounts A & B respectively
  - Consider, initially A = 1000, & B = 0
  - Transaction to transfer 500/- from A to B

Transaction (T_1)

Read (A)

A = A - 500;

Write (A)

Read (B)

B = B + 500

Write (B)

Commit

These two opn are not important from database Point of view

Transaction Log

Start : T_1

(T_1, A, 1000, 500)

If transaction T_1 fails then we need to rollback the value of dataitem A to '1000'

Consider the following representation

(T_i, X, V_1, V_2)

- T_i : Transaction number
- X : dataitem
- V_1 : old value of dataitem
- V_2 : new value of dataitem



Topic : Transaction log

* Transaction log is used to record the activities performed by the transactions
  * We can recover from a failure only if "transaction log" is available
    ↳ ∴ Transaction log is maintained by recovery management Component in the secondary memory (disk).

Note: Either we can maintain a seperate transaction log for each ongoing transaction (or) we can maintain a unified transaction log

Consider: (T_i, X, V_1, V_2)

- T_i → Transaction no.
- X → data item
- V_1 → Old value
- V_2 → new value

and, Consider the following "transaction log"

T_1 : Start
(T_1, A, 200, 120)
(T_1, B, 100, 150)
T_2 : Start
(T_2, C, 250, 75)
Check point → The operations Performed before the check point have been updated into database
(T_1, D, 90, 140)
(T_2, E, 500, 290)
T_2 : Commit
T_3 : Start
(T_3, F, 280, 100)

Let System Fails at this point!

→ T_1 & T_3 were the ongoing transactions when failure occured {i.e. they failed before Commit}
↳ we need to undo all the opn Performed by T_1 & T_3

If there is any transaction that Committed before the last check point then we don't need to worry about those transaction

If any transaction/s Committed after the last check point, then we need to "redo" only those operations of that transaction that appeards after last check point



2 mins Summary

- **Topic** Transaction
- **Topic** ACID properties
- **Topic** Atomicity
- **Topic** [redacted]

Slide

THANK - YOU